TAUSIF

React: Project: Weather App

Build-along · API calls · React Query · Tailwind · deploy



Md Tausif Hossain

Your React Instructor

Tech Lead @ DevTechGuru · Founder, **TechnicalBind**

9+ years shipping production software

Instructor @ **Ostad**: ostad.app/instructor/73688

🌐 tausif.bd · GitHub · LinkedIn

What we'll cover

Project **goal** & final feature list

Setup: Vite, Tailwind, React Query, API key in `.env`

An **API module** that fetches and **shapes** the response

A **custom hook** over `useQuery` for caching, loading & errors

State design + a controlled search form

Rendering the **weather card**, then **bonus features** and **deploy**

The Project

What we're building

Weather Watch — search any city, see the current conditions.

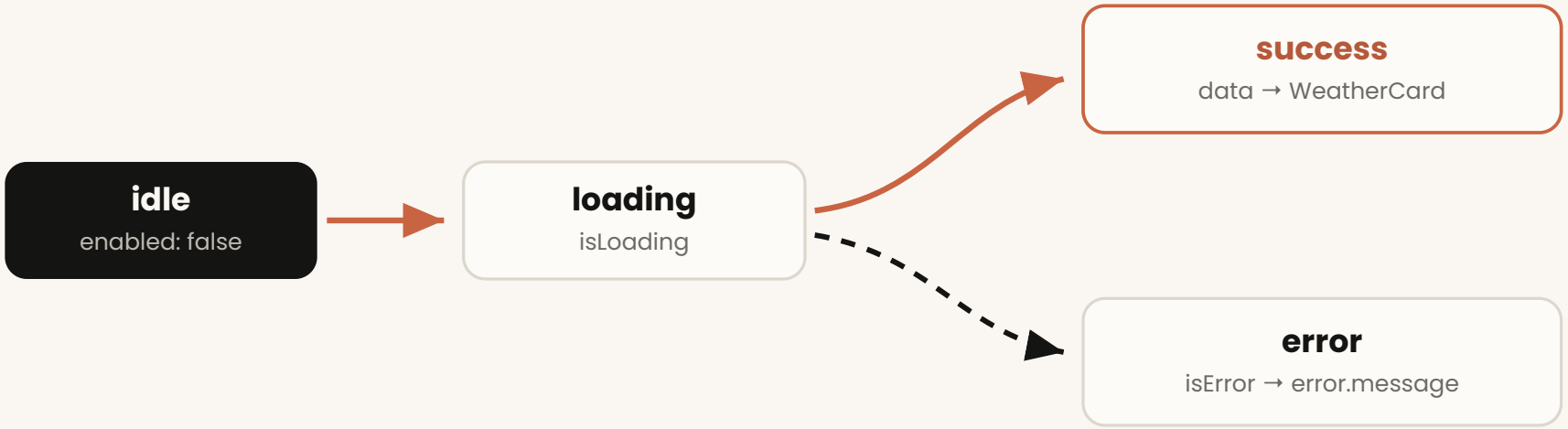
- 🔍 **Search a city** by name
- 🌡️ **Current weather**: temperature, condition, humidity, wind
- ⏳ **Loading** line while the request is in flight
- ⚠️ **Error** message for a bad city name or a network failure
- 💾 **Cached** results — searching a city twice costs one request
- ✨ Bonus: **geolocation**, **°C/°F toggle**, **5-day forecast**

How it flows

```
User types "Dhaka" → submit
   → setCity("Dhaka")          → useWeather(city) fires
   → isLoading  → data?        → render WeatherCard
                → isError?     → render the error line
```

> We'll build it bottom-up: API module → hook → state → form → UI.

The four UI states



React Query models all four — you only choose **what to render**

> Every search walks this path: idle, then loading, then either success or error. Exactly one branch renders at a time.

Setup

Scaffold with Vite

```
pnpm create vite weather-watch --template react
cd weather-watch
pnpm install
pnpm dev                        # http://localhost:5173
```

- Vite gives an instant dev server + fast HMR.
- `npm` or `yarn` work the same — swap the command.

Add React Query & Tailwind

```
pnpm add @tanstack/react-query
pnpm add -D tailwindcss postcss autoprefixer
npx tailwindcss init -p      # writes tailwind.config.js + postcss.config.js
```

- **React Query** owns fetching, caching, loading and error state.
- **Tailwind** gives utility classes — no CSS files to name and maintain.

Wire up Tailwind

```
// tailwind.config.js
export default {
  // Scan these files for class names so Tailwind keeps only what you use.
  content: ['./index.html', './src/**/*.{js,jsx}'],
  theme: { extend: {} },
  plugins: [],
}
```

```
/* src/index.css */
@tailwind base;
@tailwind components;
@tailwind utilities;
```

Folder structure

```
src/
  api/
    weatherApi.js      # HTTP lives here
  hooks/
    useWeather.js      # React Query wrapper
  components/
    SearchBar.jsx
    WeatherCard.jsx
  App.jsx              # state + which branch to render
  main.jsx             # QueryClientProvider mounts here
  index.css            # Tailwind directives
.env                   # API key (never commit!)
.gitignore
```

> Three layers: **api/** talks HTTP, **hooks/** handles caching, **components/** only render.

Get a free API key

1. Sign up at **openweathermap.org/api** (free tier).
2. Copy your API key from the dashboard.
3. New keys can take **a few minutes to hours** to activate: don't panic on early 401s.

> We'll use the **Current Weather** endpoint: `api.openweathermap.org/data/2.5/weather`.

Store the key in .env

Vite only exposes variables prefixed with `VITE_` to the browser:

```
# .env
VITE_WEATHER_API_KEY=your_key_here
```

```
const API_KEY = import.meta.env.VITE_WEATHER_API_KEY
```

> Gotcha: forget the `VITE_` prefix and `import.meta.env` returns `undefined`. Restart the dev server after editing `.env`.

Protect the key in .gitignore

```
# .gitignore
node_modules
dist
.env
```

⚠️ A frontend `.env` key is still shipped to the browser: anyone can read it in DevTools. For a class demo that's fine; in production, proxy the call through a backend so the key stays secret. **Never commit `.env`.**

API Service Module

api/weatherApi.js — the request

```
const BASE_URL = 'https://api.openweathermap.org/data/2.5/weather'
const API_KEY = import.meta.env.VITE_WEATHER_API_KEY

export async function getWeather(city) {
  // units=metric → Celsius, wind in m/s. encodeURIComponent handles "New York".
  const url = `${BASE_URL}?q=${encodeURIComponent(city)}&units=metric&appid=${API_KEY}`
  const response = await fetch(url)

  // fetch does NOT throw on 404/401 — we must check ok ourselves.
  if (!response.ok) {
    if (response.status === 404) throw new Error('City not found. Check the spelling.')
    throw new Error('Could not fetch weather. Try again.')
  }
```

> Translate HTTP status into friendly errors **here**, so components stay simple.

api/weatherApi.js — shape the response

```
  const data = await response.json()

  // Return only the fields we need, in a friendly shape.
  return {
    city: data.name,
    temperature: Math.round(data.main.temp),
    condition: data.weather[0].main,
    humidity: data.main.humidity,
    windSpeed: Math.round(data.wind.speed * 3.6),    // m/s → km/h
  }
}
```

> The raw response nests everything (`main.temp`, `weather[0].main`, `wind.speed`). Normalise once here and no component ever learns the API's shape.

Data Layer: React Query

main.jsx — the provider

```
import { QueryClient, QueryClientProvider } from '@tanstack/react-query'

// One QueryClient for the whole app. It stores the cache React Query uses.
const queryClient = new QueryClient()

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <QueryClientProvider client={queryClient}>
      <App />
    </QueryClientProvider>
  </React.StrictMode>,
)
```

> Forget the provider and every `useQuery` throws "No QueryClient set".

hooks/useWeather.js

```
import { useQuery } from '@tanstack/react-query'
import { getWeather } from '../api/weatherApi'

// Components call this and get { data, isLoading, isError, error }.
export function useWeather(city) {
  return useQuery({
    queryKey: ['weather', city],       // cache identity, per city
    queryFn: () => getWeather(city),   // what actually fetches
    enabled: !!city,                   // no city → no request
    retry: false,                      // don't retry a 404 forever
  })
}
```

The four options that matter

- `queryKey` — the cache address. Same key = same cached data.
- `queryFn` — the async function; it must **throw** on failure.
- `enabled` — `!!city` turns `''` into `false`, so we don't fetch on an empty search.
- `retry` — `false`, because "city not found" will never succeed on retry.

> Anything in the key that changes triggers a refetch. That's how the °C/°F toggle works later.

What the hook hands back

```
const { data, isLoading, isError, error } = useWeather(city)
```

- No `useState` for loading or error — React Query tracks both.
- No `try / catch / finally` in the component.
- No `useEffect` — the hook fetches when the key changes.
- Re-searching a city you already looked up is **instant**: it's cached.

State & Form

State design

One piece of state in `App` covers the whole flow:

```
export default function App() {
  // The city we are currently showing weather for.
  const [city, setCity] = useState('')

  // Ask React Query for this city's weather.
  const { data, isLoading, isError, error } = useWeather(city)
}
```

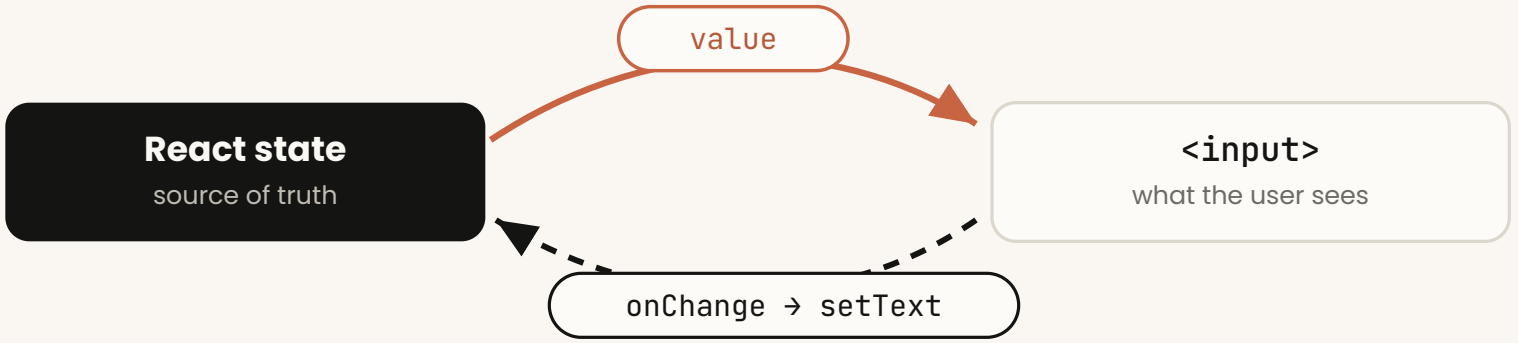
> **Server state** (the weather) belongs to React Query. **UI state** (the searched city) belongs to `useState`. Never copy one into the other.

Controlled search form

```
export function SearchBar({ onSearch }) {
  const [text, setText] = useState('')      // whatever the user is typing

  function handleSubmit(event) {
    event.preventDefault()                  // stop the page from reloading
    const city = text.trim()
    if (!city) return                       // ignore empty searches
    onSearch(city)                          // hand the city up to App
  }
  return (
    <form onSubmit={handleSubmit} className="flex gap-2">
      <input value={text} onChange={(e) => setText(e.target.value)} />
      <button type="submit">Search</button>
    </form>
  )
}
```

Controlled input loop



Controlled input: state is the **single source of truth**

> The input's value comes from state, every keystroke calls `setText`, and React re-renders with the new value.

Rendering

App.jsx — render the right state

```jsx
<div className="min-h-screen bg-blue-50 flex justify-center p-4">
  <div className="w-full max-w-md mt-16 space-y-4">
    <h1 className="text-center text-3xl font-bold text-blue-600">Weather Watch</h1>

    {/* On submit, save the city → triggers the query. */}
    <SearchBar onSearch={setCity} />

    {isLoading && <p className="text-center text-gray-500">Loading...</p>}
    {isError && <p className="text-center text-red-500">{error.message}</p>}
    {data && <WeatherCard data={data} />}
  </div>
</div>
```

> Mutually exclusive branches: loading **or** error **or** data. Before the first search, `enabled: false` keeps all three quiet — you just see the form.

WeatherCard.jsx

```jsx
// Pure presentation: no state, no fetch.
// data → { city, temperature, condition, humidity, windSpeed }
export function WeatherCard({ data }) {
  return (
    <div className="rounded-2xl bg-white p-6 shadow-md">
      <h2 className="text-2xl font-bold text-gray-800">{data.city}</h2>
      <p className="text-gray-500">{data.condition}</p>

      <p className="my-4 text-5xl font-bold text-blue-500">{data.temperature}&deg;C</p>

      <div className="flex justify-between text-gray-600">
        <span>Humidity: {data.humidity}%</span>
        <span>Wind: {data.windSpeed} km/h</span>
      </div>
    </div>
  )
}
```

Tailwind styling notes

- The card is four utilities: `rounded-2xl bg-white p-6 shadow-md`.
- Centre the column with `flex justify-center` + `max-w-md`.
- `space-y-4` on the wrapper spaces every child — no per-element margins.
- Keep the temperature the visual anchor: `text-5xl font-bold`.

> Utilities read as noise at first and as a design system by the third component. Resist writing a CSS file.

Bonus Features

Geolocation: local weather

```
// api/weatherApi.js — same endpoint, lat/lon instead of q
export async function getWeatherByCoords(lat, lon) {
  const url = `${BASE_URL}?lat=${lat}&lon=${lon}&units=metric&appid=${API_KEY}`
  // ...same ok-check and shaping as getWeather
}

// App.jsx — ask once on first load, then let the query key do the rest
useEffect(() => {
  navigator.geolocation.getCurrentPosition(
    ({ coords }) => setCoords(coords),
    () => setGeoDenied(true),          // always handle the denied callback
  )
}, [])
```

> `getCurrentPosition` is async and needs HTTPS (localhost is allowed).

Unit toggle (°C / °F)

```
const [unit, setUnit] = useState('metric')   // 'metric' | 'imperial'

<button onClick={() => setUnit((u) => (u === 'metric' ? 'imperial' : 'metric'))}>
  {unit === 'metric' ? '°C' : '°F'}
</button>

// hooks/useWeather.js — unit becomes part of the cache identity
queryKey: ['weather', city, unit],
queryFn: () => getWeather(city, unit),
```

> Add `unit` to the **queryKey** and React Query refetches on its own — and keeps °C and °F cached separately.

Bonus: 5-day forecast

```
// api/weatherApi.js
export async function getForecast(city) {
  const url = `${FORECAST_URL}?q=${city}&units=metric&appid=${API_KEY}`
  // ...same ok-check, then return the shaped list
}

// hooks/useForecast.js — a second hook, same pattern
export function useForecast(city) {
  return useQuery({
    queryKey: ['forecast', city],
    queryFn: () => getForecast(city),
    enabled: !!city,
  })
}
```

`/forecast` returns 3-hour steps; pick one entry per day (e.g. midday) to show 5 cards.

Deploy

Build & deploy

```
pnpm build      # outputs dist/
pnpm preview    # test the production build locally
```

- **Vercel / Netlify**: connect the repo, framework auto-detected as Vite.
- Add `VITE_WEATHER_API_KEY` in the host's **Environment Variables** dashboard — it isn't in your repo.

> Gotcha: forgetting the env var on the host = a deployed app that 401s. Set it before the first deploy.

🧪 Build it (live, 20 min)

1. Scaffold Vite + Tailwind, add your `.env` key (with `VITE_` prefix).
2. Write `api/weatherApi.js`: fetch, map errors, shape the response.
3. Wrap the app in `QueryClientProvider`.
4. Write `hooks/useWeather.js` with `enabled` + `retry: false`.
5. Add the `city` state and the controlled `SearchBar`.
6. Render loading / error / `WeatherCard` as exclusive branches.
7. Bonus: geolocation **or** a °C/°F toggle.

Recap

Keep HTTP in **api/**; map status codes to friendly errors and **shape** the response there

Store the key in `.env` with the `VITE_` prefix; gitignore it (still public in the browser)

React Query owns server state: `queryKey`, `queryFn`, `enabled`, `retry`

`useState` holds one thing — the searched **city**; the hook returns the rest

Render **loading / error / data** as mutually exclusive branches

Bonus: geolocation, unit toggle, forecast; deploy to Vercel/Netlify with the env var set

Let's connect

Questions after class? Reach out any time.

🌐 Portfolio: **tausif.bd**

💼 LinkedIn: linkedin.com/in/tausif1337

💻 GitHub: github.com/tausif1337

💬 WhatsApp: wa.me/8801748181448

Keep building. See you next session.